

ZAK MCP Gateway v0.3

How it works, in plain words: every technique, a picture of it, an everyday comparison, and the measured results.

95.5%

fewer input tokens over a whole 8-task agent run vs no gateway (best case)

85%

smaller tool definitions on every model request (3,600 → 541)

96.6%

less tool output reaching the model (221,600 → 7,441)

–35%

fewer tokens than v0.2 on the same tasks (best case)

Measured on 5 Oct 2026 against the real GitHub MCP server (26 tools), public repo modelcontextprotocol/servers, o200k tokenizer, same calls in all setups. Details and limits in section 7.

1. The problem in one minute

An AI agent uses tools (GitHub, files, databases, ticket systems) through **MCP**. The model is billed per **token**, roughly a short piece of a word. MCP wastes tokens in three places:

The manuals. Every tool's full JSON description goes with every request. GitHub alone is 3,600 tokens; a 227-tool company server is about 77,000.

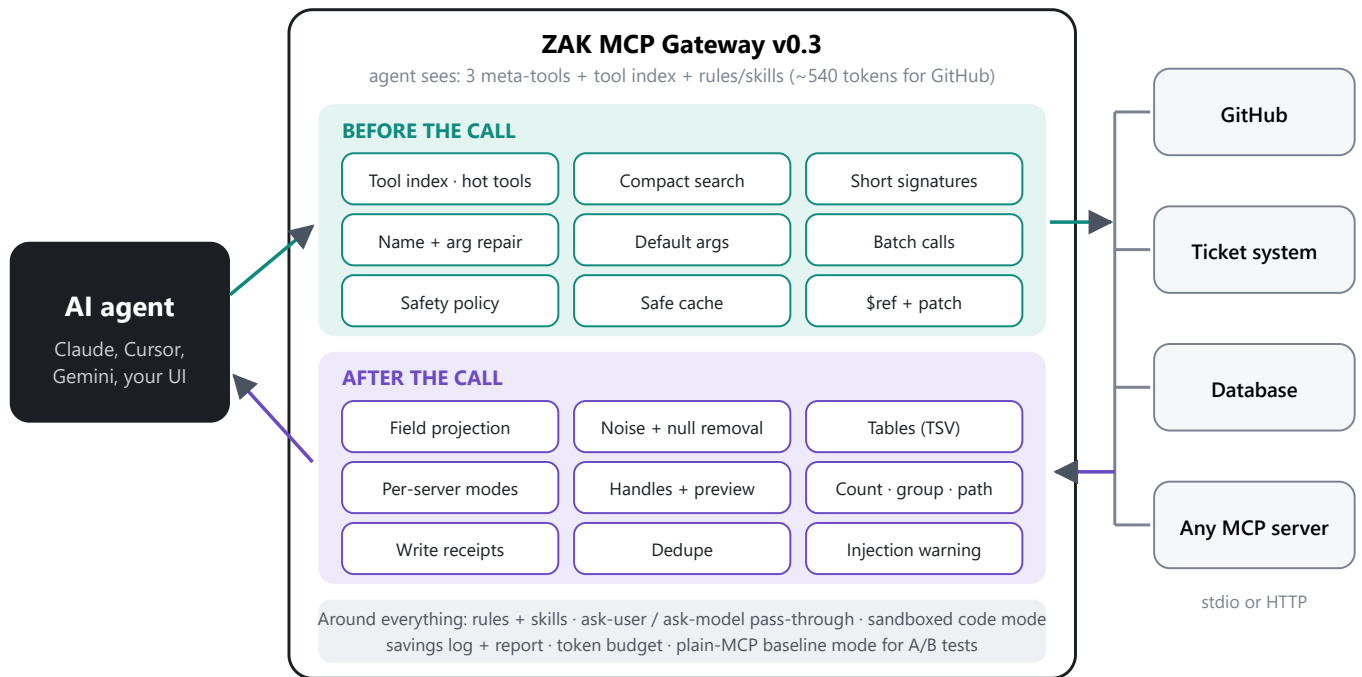
The results. Asking for 10 commits returns ~18,000 tokens of JSON (avatars, API links, IDs). The agent needed ~800.

The turns. Every tool call is another model request that re-sends the whole conversation. Extra searches and paging multiply all of the above.

Analogy: you ask a hotel front desk for "a taxi". Before you can speak, they hand you the manual for every hotel service; when the taxi arrives they also give you the driver's employment file; and you have to walk back to the desk for every small question. The gateway is a good concierge: a one-page directory, a short answer, the file kept in the back office, and several errands handled in one trip.

2. The big picture

The gateway is one MCP server the agent connects to; all your real MCP servers sit behind it. Teal techniques shrink what goes **into** the model, purple ones shrink what comes **back**, and the grey strip makes any agent more precise and safe.



3. What is new in v0.3

v0.2 already made each answer small. Your company test showed the next problem: **extra turns**. Searches and re-paging added model requests, and each one re-sends the whole conversation. v0.3 attacks turns, keeps big data out of the conversation, and makes weaker models more precise.

Fewer turns

Call tools by name from the index; compact search; most-used tools listed up front; several calls in one batch; counts and groupings computed in the gateway.

Like a shopping list: one trip, not one trip per item.

Data stays in the gateway

Write receipts instead of full echoes; \$ref passes stored data into the next call; patches edit it in place; one field in full by path.

Like forwarding a parcel without opening it.

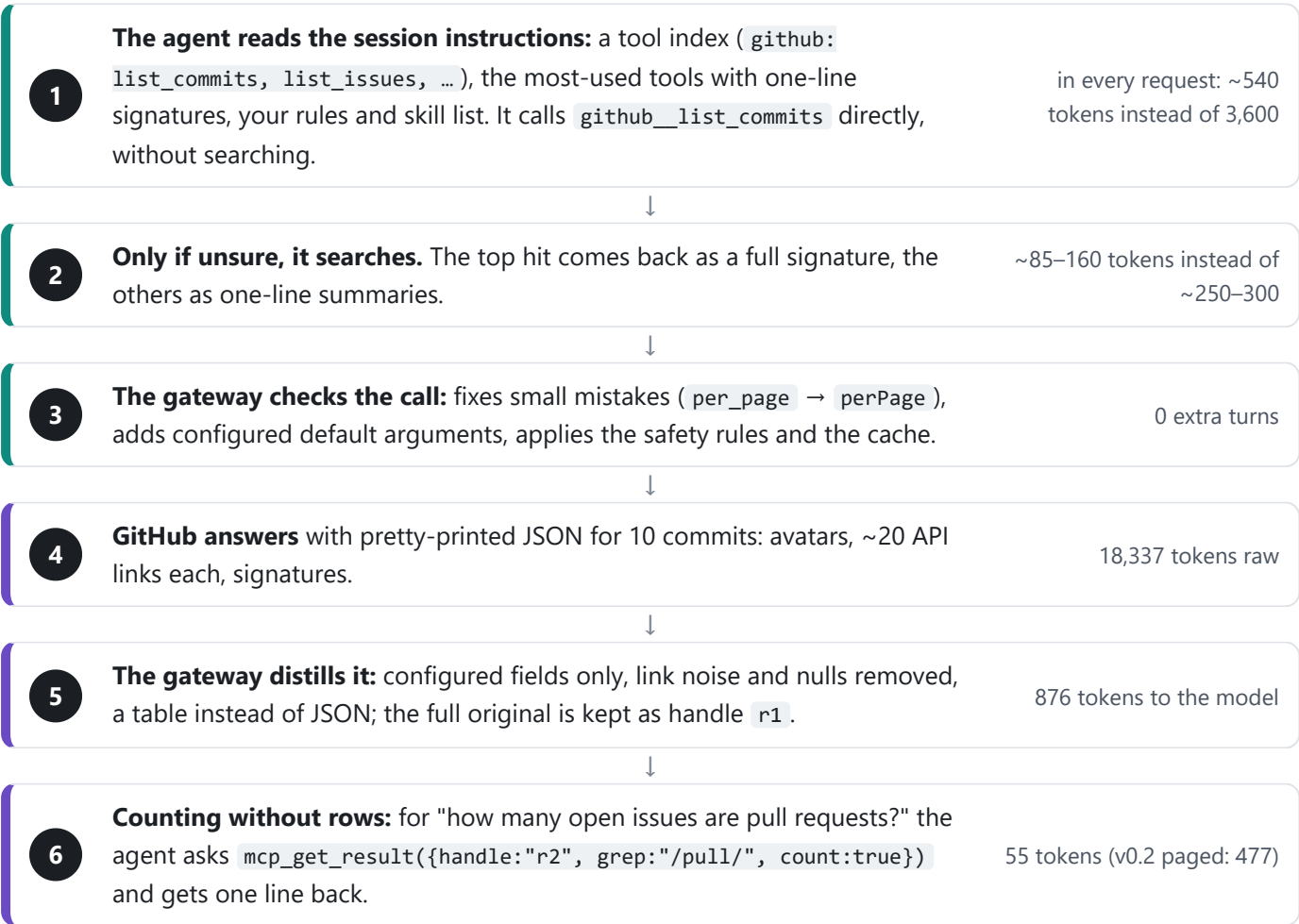
Any agent, more precise

Your rules and skills reach every client; the downstream server can ask the user or the model through the gateway; a sandboxed code mode; helpers for weak models.

Like giving every new hire the same team handbook.

4. One request, step by step

Real example from the benchmark: "show the last 10 commits of modelcontextprotocol/servers", then "how many open issues are pull requests?"



What the model receives in step 5 (columns shortened for print):

```
sha      commit.message      commit.author.name  commit.author.date  html_url
5abed86... Merge pull request #4939 from ...policy-to-main  Cliff Hall          2026-10-04T04:01:54Z https://github.com/...
a207787... chore: sync the bug report form with v2/main... cliffhall           ...                 https://github.com/...
... 8 more rows
[r1: default fields, noise keys removed; full data via mcp_get_result]
```

5. Every technique, with an everyday comparison

Technique	Where	What it does (simply)	Real-world analogy
Meta-tools	before	The agent sees 3 tools (search, call, get_result) instead of every server's tools.	One concierge desk instead of 40 counters.
Tool index new modes	before	Tool names in the instructions, as full names, families ({create,get,list}_issue), server list, or off. Sized for 20 or 200+ tools.	The lobby directory: detailed or by floor.
Hot signatures new	before	One-line signatures of the most-used tools (from the stats log) in the instructions, so the agent calls them without searching.	The popular dishes printed on the front of the menu.
Compact search new	before	BM25 search with synonyms; top hit as a full signature, the rest as name + summary. Servers can add their own words (ticket → incident).	A shortlist from the librarian, not the whole catalog.
Short signatures	before	A long JSON schema becomes one TypeScript-style line with parameter hints; too-complex schemas keep the full minified version.	A recipe card instead of the cookbook chapter.
Name + argument repair	before	Accepts <code>github::x</code> , plurals, <code>per_page</code> for <code>perPage</code> , <code>"5"</code> for 5; real mistakes return the signature.	A clerk who fixes a misspelled street.
Default arguments new	before	Per tool, the config fills in page size or server-side field selection when the agent leaves it out, so heavy data is never fetched.	A standing order: "the usual, small portion".
Batch calls new	before	Several calls in one turn, in order, one result line each. Writes are previewed first and run after one confirmation; stops at the first error and says where.	One trip with a full shopping list.
Safety policy	before	Read-only modes, allow/deny lists, writes need <code>confirm: true</code> ; read/write from MCP annotations, verbs, or per-tool overrides; unknown = write.	A teller asking for your signature.
Safe cache upgraded	before	Repeat reads from memory; a call becomes a write when its arguments say so (<code>writeIf</code>); servers on one backend share invalidation; per-server TTL or off; <code>fresh:true</code> .	Updating every copy of the menu when one changes.
Field projection	after	Only the fields asked for, or per-tool defaults; works through lists.	A highlighter on a long report.
Noise + null removal, tables	after	Drops API links, internal IDs, empty values; lists become tables when smaller.	Junk mail out; a spreadsheet, not a letter per row.
Per-server modes new	after	<code>full</code> for raw-JSON servers, <code>light</code> for already-compact servers (no reshaping, bigger limit), <code>off</code> .	Don't re-pack what is already packed.
Handles + preview	after	Big results stay in the gateway; the model gets the first rows and a handle (<code>r3</code>) to page, grep or pick fields.	A coat-check ticket.

Count, group, path new	after	<code>count</code> , <code>group_by</code> , <code>distinct</code> , <code>sort</code> on a stored result; <code>path</code> returns one field in full (e.g. a file's content).	Asking the clerk for the total, not every receipt.
Write receipts new	after	<code>{ok, id, ...}</code> instead of the server's full echo; the echo stays behind a handle.	A receipt, not a copy of the whole order.
Dedupe new	after	A byte-identical repeat result becomes "unchanged, same as r5".	"Same as yesterday."
Injection warning	after	Results with text like "ignore previous instructions" are flagged as untrusted.	An "unverified sender" stamp.
\$ref + patch new	around	An argument can point at stored data (<code>{"\$ref": "r3", "path": "content"}</code>), even from another server, with find/replace edits. Fails loudly on a missing handle or an ambiguous edit.	Forwarding a parcel without opening it.
Rules + skills new	around	Your rules go into every session; skills are listed by name, loaded with <code>mcp_get_skill</code> , and suggested by search when they match.	The team handbook for every assistant, not just one.
Ask-user / ask-model pass-through new	around	When a downstream server needs to ask the user (elicitation) or the model (sampling), the gateway forwards it to the agent's client. Off by default.	Putting a call through to the right desk.
Sandboxed code mode upgraded	around	A short script calls several read tools; only the answer returns. Runs in a separate process with no file access, no child processes, a memory cap and a hard timeout. Off by default.	A locked workroom with one phone line.
Savings log, report, budget	around	Every call logged; <code>npm run report</code> shows savings, turns by kind and suggested default fields; optional session budget.	An electricity meter with a prepaid limit.
Plain-MCP baseline	around	<code>--passthrough</code> runs the same servers as plain MCP for fair A/B tests.	Weighing yourself on the same scale.

6. Helping weaker models (agent UI)

Small models (Gemini Flash, Claude Haiku) lose precision when they must pick among many tools, follow long rules, plan many steps and read big results. The rule is: **move decisions out of the model and into code**. The test agent UI now has four switches that do this:

Switch	What it does	Analogy
Preload tools + skill	Searches the first question up front and puts the best signatures and any matching skill in the prompt.	A waiter who brings water before you ask.
Focused tools	For each question, the 5 best-matching tools become real functions, so the model calls them directly instead of search-then-call.	Laying out only the tools for today's job.
Trim old results	Tool results from earlier turns are shortened in the history (with their handle), so they stop being re-sent in full every turn.	Filing old paperwork instead of carrying it around.
Verify the answer	IDs, SHAs and long numbers in the answer must appear in the tool results; otherwise the model gets one chance to correct itself.	A supervisor checking the figures before the report ships.

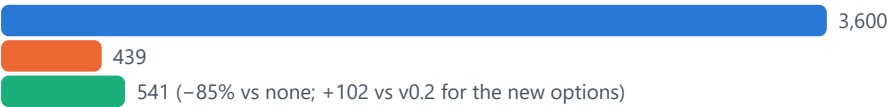
Live result (Gemini, focused + trim + verify on): "last 3 commits" used 2 model calls and called the right tool with the right arguments on the first try. A harder counting question took 47k input tokens in 10 calls, against about 110k in 12 calls for a similar question on the old gateway. The run also exposed one real gap (a `path` meant per row), which is fixed and now has a test.

7. Measured results: no gateway vs v0.2 vs v0.3

Live, real GitHub MCP server (26 tools), public repo, 8 read-only tasks, same calls in all three setups, o200k tokenizer. Command: `npm run bench:compare -- <v0.2 dist>`.

No gateway v0.2 v0.3 (solid = best case, outline = worst case) each panel has its own scale

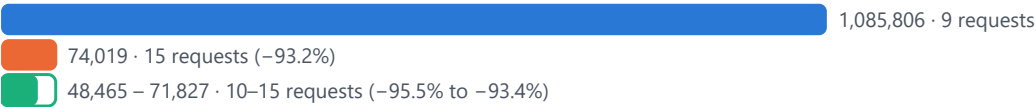
Tool definitions per model request



Tool output reaching the model (incl. searches and paging)



Whole 8-task agent run: input tokens (every request re-sends definitions + history)



v0.3 best case: the agent calls tools by name, as its instructions say. Worst case: it still searches before every new tool (with compact results).

Task	No gateway	v0.2	v0.3	What made the difference
list_commits (10)	18,337	876 + 84	876 + 84	default fields → table
list_commits (30), sha + message	61,589	1,280	1,280	projection → table
list_issues (20 open)	37,846	909 + 296	909 + 157	compact search
list_pull_requests (10)	27,375	839 + 258	839 + 223	compact search
search_repositories (10)	3,470	485 + 287	485 + 144	compact search
README	2,498	1,212 + 224	1,212 + 133	compact search
list_commits again	18,337	876	876	cache (no GitHub call)
"How many of 30 open issues are PRs?"	52,148	909 + 477	909 + 55	count instead of paging

" + n" = tokens spent on searching or follow-up reads. Agent-run totals add every model request: tool definitions + all earlier history + 40 tokens per tool call written by the model.

Read these numbers honestly

- **The best case needs the model to cooperate.** v0.3 tells the agent to call tools by name; if it keeps searching anyway, v0.3 is only ~3% better than v0.2 on this task set.
- **v0.3's definitions are ~100 tokens bigger** than v0.2's (batch, `$ref`, counting options). The saving comes from fewer and smaller turns.
- **Read-only benchmark.** Write receipts, batch writes, `$ref` /patch, history trimming and skills don't show on a public read-only repo; 55 automated tests cover them.
- **Small setups can cost more.** With a handful of small tools the gateway's definitions exceed direct MCP's. The gateway pays off with many tools (your 227-tool server: ~77k → ~1.5k tokens per request) or big results.
- **Inside Claude Code**, MCP is a small share of each request (about 2.6% in your measurements), so the whole-session saving there is small. The gains are largest for MCP-heavy agents such as your user UI.
- **Tokens are not task success.** The o200k tokenizer stands in for Claude/Gemini; use the agent UI's A/B mode for real-model numbers (one GitHub question: 20,806 → 2,522 input tokens, -87.9%).

8. Bugs found and fixed

Problem	Effect	Fix
v0.2		
Cache key built with <code>JSON.stringify(obj, keyArray)</code>	Every call hashed to the same key	Stable, sorted serialization; regression test
Caching never on; results double-encoded	No cache hits; results bigger than raw	Read/write classifier; return distilled text directly
<code><YOUR_GITHUB_TOKEN></code> placeholder	Overrode the real token → GitHub 401	Placeholders ignored; <code>\${VAR}</code> secrets
<code>github::</code> vs <code>github__</code> ; <code>per_page</code> vs <code>perPage</code>	"Tool not found"; setting silently ignored	One separator, tolerant lookup, argument repair
Agent UI "tokens saved" was a made-up formula	Misleading numbers	Real A/B with the model's own usage numbers
v0.3 (found by tests and live runs)		
Already-compact replies re-paged under one global limit	Extra turns on compact servers	Per-server limits and <code>light</code> mode
Raw paging cut long lines without saying so	The agent thought it had everything	Header says CLIPPED, and how to continue
Downstream server's own instructions dropped	Server guidance lost behind the gateway	Forwarded (capped), switchable per server
Grouped catalog format hid where the verb goes	Models would guess wrong names	<code>{create,get,list}_issue</code> notation + plural-tolerant lookup
Gemini sent <code>path</code> meaning "this field of each row"	Errors, 3 wasted turns in a live run	Falls back to a per-row field; test added
Answer verifier matched "1234" inside "INC0012345"	Wrong values could pass the check	Whole-token matching; test added

9. Plan coverage

Plan	Status	Notes
Build plan phases 1–4 (core, results, stats, safety)	done	Since v0.2.
Build plan phase 5 (benchmark)	partial	GitHub 3-way benchmark and live Gemini runs done; a 10–20 task real-model run over 3+ servers with success rates is still open.
Build plan phase 6 (advanced)	mostly	Sandboxed code mode, safe cache, tool preload, suggested default fields done; a cheap-model router belongs in the host app.
Economy plan: 14 ideas	all done	Call-first + hot signatures, compact search, batch, per-server limits, field by path, receipts, synonyms, safe cache, safer code mode, dedupe + stats, catalog modes, <code>\$ref</code> , patch, suggested fields.
Extra ideas	done	Forwarded server instructions, default-argument pushdown, aggregation on handles, ask-user/ask-model pass-through, history trimming, live-model benchmarking.
Weak-model precision plan	started	Rules + skills serving, focused tools, verifier, skill preload done. Still open: evaluation set, workflow tools, name → ID resolution, escalation.

10. Try it yourself

```
npm install && npm test
npm run test:github
npm run bench:compare -- <v0.2 dist dir>
npm run report
cd agent-ui && npm run dev

# 55 tests: unit, MCP round trips, v0.3 features, agent-UI helpers
# live: CLI over stdio → GitHub MCP (no token needed for public repos)
# live 3-way benchmark: no gateway vs v0.2 vs v0.3
# savings, turns by kind, suggested default fields
# Gemini agent: A/B mode + weak-model switches
```